

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)
Факультет компьютерных наук
Кафедра программирования и информационных технологий

*Разработка микросервисной платформы цифровой библиотеки с ИИ
ассистентом*

Курсовая работа по дисциплине
«Учебная практика (научно-исследовательская работа)»
09.03.02 Информационные системы и технологии
Профиль «Программная инженерия в информационных системах»

Зав. кафедрой _____ д.ф.-м.н., доцент С.Д. Махортов
Обучающийся _____ ст. 3 курса П.С. Скибин
Руководитель _____ ассистент В.А. Ушаков

Воронеж 2026

СОДЕРЖАНИЕ

ИСПОЛЬЗУЕМЫЕ ТЕРМИНЫ.....	5
ВВЕДЕНИЕ.....	8
1 Предметная область и проектирование системы.....	9
1.1 Постановка задачи и цели работы.....	9
1.2 Анализ существующих решений.....	10
1.2.1 ЛитРес.....	10
1.2.2 Bookmate.....	11
1.2.3 Goodreads.....	11
1.2.4 MyBook.....	11
1.2.5 Сравнительная таблица по ключевым критериям.....	11
1.2.6 Синтез анализа и функциональные разрывы.....	12
1.3 Требования к системе.....	12
1.3.1 Функциональные требования.....	12
1.3.2 Нефункциональные требования.....	14
1.3.3 Отличительные характеристики системы.....	15
1.3.4 Функциональные границы и ограничения системы.....	15
1.4 Проектирование системы.....	16
1.4.1 Пользовательские сценарии и варианты использования.....	16
1.4.2 Архитектура микросервисной системы.....	18
1.4.3 Проектирование API Gateway и межсервисного взаимодействия.....	20
1.4.4 Проектирование базы данных.....	21
1.4.5 Проектирование AI-контура.....	22
1.4.6 Выбор технологического стека.....	23
2 Реализация системы.....	25

2.1 Организация репозитория и развёртывания	25
2.2 Реализация аутентификации и контроля доступа	25
2.3 Реализация каталога книг и классификаторов	26
2.4 Реализация социального контура: комментарии, оценки и закладки	26
2.5 Реализация хранения и потоковой передачи файлов	27
2.6 Реализация интеллектуального ассистента и RAG-контура	28
2.6.1 Процесс индексирования и поиска в RAG-пайплайне	28
2.6.2 Алгоритм работы чат-ассистента	30
2.7 Реализация системы персональных рекомендаций	31
2.8 Реализация клиентской части	32
2.9 Интеграция клиента с серверной частью	32
2.10 Реализованные паттерны пользовательского опыта	32
2.11 Детали реализации схем данных	33
3 Тестирование и оценка результатов	34
3.1 Цели и методология тестирования	34
3.2 Сценарии нагрузочного тестирования	34
3.3 Результаты нагрузочного тестирования	34
3.3.1 Производительность транзакционного контура	34
3.3.2 Производительность AI-контура	35
3.3.3 Ресурсная утилизация	35
3.4 Анализ узких мест и компромиссов	36
3.5 Функциональное тестирование и валидация RAG	36
3.6 Интеграция мониторинга в процесс разработки	37
3.7 Итоговые выводы по разделу	37
ЗАКЛЮЧЕНИЕ	38

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	40
ПРИЛОЖЕНИЕ А.....	42

ИСПОЛЬЗУЕМЫЕ ТЕРМИНЫ

API Gateway – Единая точка входа (application programming interface gateway). Архитектурный компонент, отвечающий за маршрутизацию клиентских запросов к соответствующим микросервисам.

BM25 (Best Match 25) – Статистический алгоритм ранжирования документов на основе частотности терминов. Применяется для лексического (полнотекстового) поиска по ключевым словам в рамках RAG-контура.

CI/CD – Continuous Integration / Continuous Delivery (непрерывная интеграция и непрерывная доставка). Набор практик и инструментов для автоматизации сборки, тестирования и развёртывания программного обеспечения.

CORS – Cross-Origin Resource Sharing (механизм контроля доступа между доменами). Стандарт веб-безопасности, регулирующий взаимодействие фронтенд-приложения с серверными API.

Embedding – Числовое векторное представление текстового фрагмента в многомерном пространстве, отражающее его смысловое содержание. Семантически близкие тексты имеют близкие эмбединги. Используется для векторного поиска в Qdrant и формирования пользовательских профилей в рекомендательной системе.

OpenFeign – Декларативный HTTP-клиент для Java, входящий в состав Spring Cloud. Позволяет описывать вызовы REST API удалённых сервисов через интерфейсы с аннотациями, скрывая детали HTTP-взаимодействия.

Rate Limiting – Механизм ограничения частоты запросов от клиента за единицу времени. Реализован на уровне API Gateway для защиты от перегрузки сервисов и злоупотребления API.

Токен (LLM) – Минимальная единица текста, обрабатываемая языковой моделью (слово, часть слова или символ). Бюджет токенов — ограничение на суммарный объём текстового контекста, передаваемого модели за один запрос.

FastAPI – Современный высокопроизводительный веб-фреймворк для создания API на языке Python, поддерживающий асинхронную обработку запросов.

FB2 (FictionBook 2) – Открытый XML-формат электронных книг, используемый в системе для хранения и потоковой передачи текстового контента произведений (MIME-тип: application/x-fictionbook+xml).

Grafana – Платформа для визуализации метрик и мониторинга, используемая в связке с Prometheus для отображения дашбордов производительности системы.

JWT – JSON Web Token (веб-токен в формате JSON). Открытый стандарт для создания токенов доступа, используемый в системе для аутентификации и авторизации пользователей.

k6 – Инструмент нагрузочного тестирования с открытым исходным кодом, позволяющий эмулировать поведение пользователей и оценивать производительность системы.

LLM – Large Language Model (большая языковая модель). Генеративная нейросетевая архитектура, выступающая в роли AI-компонента для формирования текстовых ответов на запросы пользователей.

MMR (Maximal Marginal Relevance) – Алгоритм балансировки релевантности и разнообразия результатов. Применяется в рекомендательной системе для устранения дублирования в итоговой выдаче при сохранении высокой точности.

PostgreSQL – Свободная объектно-реляционная система управления базами данных, используемая для хранения транзакционных данных микросервисов.

Prometheus – Система мониторинга и оповещения с открытым исходным кодом, собирающая метрики из различных компонентов системы.

Qdrant – Векторная база данных с открытым исходным кодом для поиска по сходству, используемая для хранения эмбеддингов в контуре RAG и рекомендаций.

RAG – Retrieval-Augmented Generation (генерация с дополнением поиском) — метод работы с языковыми моделями, при котором перед генерацией ответа выполняется поиск релевантных фрагментов во внешней базе знаний. Позволяет формировать ответы на основе актуального контекста без дообучения модели.

Redis – Высокопроизводительное хранилище данных типа «ключ–значение» в оперативной памяти. В системе используется для кэширования сессий пользователей, истории диалогов AI-ассистента и результатов рекомендаций.

REST – Representational State Transfer — архитектурный стиль проектирования распределённых систем, определяющий набор ограничений для построения API. Обеспечивает stateless-взаимодействие между клиентом и сервером через стандартные HTTP-методы.

RRF – Reciprocal Rank Fusion (метод слияния по взаимным рангам). Алгоритм объединения результатов ранжирования из нескольких поисковых контуров (например, векторного и текстового поиска) для повышения точности выдачи.

SPA – Single Page Application (одностраничное веб-приложение). Архитектура клиентской части, при которой загрузка и обновление контента происходит динамически без полной перезагрузки страницы в браузере.

Spring Boot – Фреймворк на базе Java для создания микросервисов и веб-приложений, предоставляющий готовую конфигурацию и инструменты для быстрой разработки.

Vue.js / Vue 3 – Прогрессивный фреймворк для создания пользовательских интерфейсов, используемый для реализации клиентской части приложения (SPA).

ВВЕДЕНИЕ

Электронные библиотеки эволюционируют от простых каталогов метаданных к комплексным платформам, предоставляющим встроенное чтение, сохранение прогресса и персонализированный пользовательский опыт. Современные требования включают не только доступ к контенту, но и семантический поиск, а также интеллектуальную обработку текста с помощью диалоговых AI-ассистентов. Традиционные монолитные решения часто не справляются с гетерогенностью таких нагрузок, что делает микросервисную архитектуру наиболее целесообразным подходом для построения масштабируемых библиотечных систем [1,2].

Настоящая работа посвящена решению проблемы функциональной фрагментации современных читательских сервисов. Разрабатываемая система «Цифровая библиотека» («Digital Library») объединяет классический каталог, интерфейс чтения, социальный контур и интеллектуальный AI-ассистент на базе Retrieval-Augmented Generation (RAG) [3,4] в рамках единого микросервисного веб-приложения, закрывая разрывы традиционных библиотечных решений.

1 Предметная область и проектирование системы

В данном разделе проводится контекстуализация проекта Digital Library: анализируются существующие решения, формулируются требования к системе и описывается проектирование архитектуры.

1.1 Постановка задачи и цели работы

В рамках данной работы рассматривается проблема интеграции интеллектуальных функций обработки естественного языка в классические системы электронных библиотек. Существующие на рынке решения, как правило, специализируются на узких задачах: коммерческом распространении контента, социальном взаимодействии читателей или непосредственно чтении. Традиционные платформы часто разделяют функции каталогизации, чтения и анализа, не предоставляя единого контекстного окна для глубокой работы с контентом. Это вынуждает пользователей переключаться между различными сервисами, что снижает удобство и непрерывность читательского опыта.

Целью настоящей работы является проектирование и разработка микросервисной веб-системы «Digital Library», которая объединяет управление каталогом книг, встроенный интерфейс чтения, механизмы социального взаимодействия (комментарии, оценки, закладки) и интеллектуальный поиск по содержанию на основе Retrieval-Augmented Generation (RAG) в едином приложении.

Для достижения поставленной цели в работе необходимо решить следующие задачи:

1. Провести сравнительный анализ предметной области и существующих платформ (ЛитРес, Bookmate, Goodreads, MyBook) для выявления функциональных разрывов и недостатков текущих решений.
2. Сформулировать функциональные и нефункциональные требования к системе, определяющие её масштабируемость, безопасность и производительность.

3. Спроектировать микросервисную архитектуру системы с чётким разделением доменов, единым API-шлюзом и механизмами межсервисного взаимодействия.
4. Реализовать backend-модули системы: сервисы каталога, аутентификации, социального контура и хранения файлов.
5. Разработать интеллектуальный контур на базе RAG для семантического поиска по тексту и диалогового взаимодействия, а также модуль персональных рекомендаций.
6. Реализовать клиентскую часть (Single Page Application на Vue 3) и обеспечить её интеграцию с серверными микросервисами.
7. Провести нагрузочное и функциональное тестирование системы для оценки её производительности, устойчивости и практической применимости.

Объектом исследования выступают информационные системы управления электронными библиотеками и процессы предоставления доступа к цифровому контенту.

Предметом исследования являются методы построения микросервисных архитектур и алгоритмы применения технологий Retrieval-Augmented Generation (RAG) для семантического поиска и анализа текстовых корпусов.

1.2 Анализ существующих решений

Для контекстуализации проекта Digital Library проводится сравнение с основными платформами: ЛитРес, Bookmate, Goodreads и MyBook.

1.2.1 ЛитРес

ЛитРес — коммерческая платформа распространения электронных книг и аудиокниг с каталогом, встроенным чтением, рецензиями, подпиской и синхронизацией прогресса.

Интеллектуальный анализ текста и вопросно-ответные механизмы в платформе предусмотрены в ограниченном объёме или отсутствуют.

1.2.2 Bookmate

Bookmate — платформа личной библиотеки с тематическими подборками, синхронизацией позиции и социальным взаимодействием (отзывы, оценки).

1.2.3 Goodreads

Goodreads — социальная сеть читателей с рецензиями, оценками, книжными клубами и рекомендациями на основе поведения сообщества.

1.2.4 MyBook

MyBook — подписочный сервис с встроенным чтением, сохранением позиции, рекомендациями и социальными функциями.

1.2.5 Сравнительная таблица по ключевым критериям

В таблице 1.1 приведено сравнение систем по ключевым функциональным критериям.

Таблица 1.1 — Сравнение платформ электронных библиотек

Критерий	ЛитРес	Bookmate	Goodreads	MyBook	Digital Library
Каталог книг	Да	Да	Да	Да	Да
Встроенное чтение в веб	Да	Да	Ограничено	Да	Да
Сохранение позиции чтения	Да	Да	Нет	Да	Да
Оценки и рецензии	Да	Да	Да	Да	Да
Персональная библиотека/избранное	Да	Да	Да	Да	Да
Рекомендации	Да	Да	Да	Да	Да
Интеллектуальный чат по содержанию книги	Нет	Нет	Нет	Нет	Да

Критерий	ЛитРес	Bookmate	Goodreads	MyBook	Digital Library
Семантический поиск по тексту	Огр.	Огр.	Нет	Огр.	Да

1.2.6 Синтез анализа и функциональные разрывы

Существующие платформы концентрируются на коммерческом доступе (ЛитРес), удобстве чтения (Bookmate, MyBook) или социальных функциях (Goodreads). Интеграция чтения, пользовательского взаимодействия и AI-анализа в одном приложении встречается редко.

Digital Library интегрирует каталог, чтение, социальный контур и AI-помощника в единый пользовательский поток на микросервисной архитектуре.

Именно этот разрыв между обычной библиотекой и интеллектуальной средой чтения и закрывает Digital Library.

1.3 Требования к системе

Требования разделены на функциональные (определяющие операции системы) и нефункциональные (определяющие эксплуатационные свойства).

1.3.1 Функциональные требования

На уровне пользовательских операций система предусматривает: регистрацию и вход, просмотр и поиск книг, встроенное чтение, управление избранным и закладками, добавление комментариев и выставление оценок, получение персональных рекомендаций и взаимодействие с диалоговым ассистентом.

Таблица 1.2 — Функциональные требования к системе

Функциональная область	Требование
Пользователь и доступ	Система должна обеспечивать регистрацию пользователей, вход через учётные данные, завершение сессии, получение профиля и ролевое

Функциональная область	Требование
	разграничение доступа (пользователь, модератор, администратор).
Каталог и метаданные	Система должна хранить метаданные книг (название, авторы, жанры, теги, описание), предоставлять операции поиска и фильтрации по названию, автору, жанру и рейтингу.
Чтение	Система должна предоставлять доступ к содержанию книг в формате FB2, синхронизировать позицию чтения пользователя, сохранять прогресс и фиксировать время, проведённое за чтением.
Социальное взаимодействие	Система должна поддерживать добавление, редактирование и удаление комментариев к произведениям, выставление оценок, создание закладок и управление избранным.
Администрирование	Система должна предоставлять операции загрузки книг и обложек, создания и редактирования карточек произведений, а также инициирования обработки контента для интеллектуальных функций.
Интеллектуальные функции	Система должна обрабатывать текстовые запросы пользователя к содержанию книг, формировать ответы с указанием релевантных источников и предоставлять рекомендации произведений на основе истории пользователя.
Межсервисная интеграция	Система должна маршрутизировать клиентские запросы через единую точку входа (API Gateway), обеспечивать проверку целостности связей между доменами и гарантировать согласованность данных при операциях записи.

1.3.2 Нефункциональные требования

Нефункциональные требования определяют эксплуатационные свойства системы, обеспечивающие её надёжность и масштабируемость.

Таблица 1.3 — Нефункциональные требования к системе

Нефункциональная область	Требование
Архитектура	Система должна быть построена по модели микросервисов с чётким разделением доменов и единым входом (API Gateway) для клиентского доступа.
Данные	Данные каждого сервиса должны храниться в отдельной базе данных; межсервисные связи должны проверяться в бизнес-логике.
Безопасность	Защищённые операции должны требовать аутентификации посредством JSON Web Token (JWT); административные действия должны ограничиваться ролями (модератор, администратор).
Производительность	Система должна обеспечивать время отклика (p95) не более 650 мс для транзакционных операций и не более 9,5 с для интеллектуальных запросов (RAG, рекомендации). Частота ошибок уровня 5xx не должна превышать 2% при нагрузке до 50 RPS.
Надёжность	Система должна корректно работать при частичной недоступности отдельных компонентов; входной уровень должен содержать защиту от каскадных отказов.
Наблюдаемость	Система должна предоставлять метрики производительности, поддерживать централизованный мониторинг (Prometheus/Grafana) и логирование значимых событий.
Развёртывание	Система должна развёртываться через Docker Compose в локальных и тестовых средах; сборка и доставка должны автоматизироваться (CI/CD-конвейеры).

1.3.3 Функциональные границы и ограничения системы

Текущая версия системы не включает издательский контур, расширенную модерацию, офлайн-режим, нативные мобильные клиенты и коммерческую подсистему.

Таблица 1.4 — Границы текущей реализации системы

Область ограничения	Что не входит в текущий scope
Функциональное расширение	Редактирование/создание книг внутри системы, расширенная модерация контента, полноценный биллинг и коммерческий контур.
Клиентские платформы	Отдельное нативное мобильное приложение и офлайн-режим чтения как полноценный сценарий.
Интеграционные сценарии	Мгновенная мультиплатформенная синхронизация в режиме реального времени для всех клиентов.
Эксплуатационные факторы AI	Гарантированно одинаковое время отклика с транзакционными сервисами и независимость качества ответа от полноты индекса.

1.4 Проектирование системы

Архитектурное решение направлено на разделение ответственности между микросервисами, обеспечивая независимое масштабирование, упрощённое сопровождение и изоляцию точек отказа.

1.4.1 Пользовательские сценарии и варианты использования

Пользовательские сценарии охватывают полный жизненный цикл взаимодействия с системой: аутентификацию, поиск произведения, чтение, социальную активность и получение персонализированной выдачи. Шесть ключевых сценариев определяют функциональные границы системы и служат основой для проектирования архитектуры.

Таблица 1.5 — Пользовательские сценарии системы

Сценарий	Описание	Результат
П1. Аутентификация	Пользователь регистрируется или входит в систему, получает JWT-токен и доступ к защищённым функциям.	Токен сохранён; доступны операции, требующие аутентификации.
П2. Поиск произведения	Пользователь переходит в каталог, применяет фильтры и поиск,	Выбрано конкретное произведение для чтения или взаимодействия.

Сценарий	Описание	Результат
	открывает карточку интересующей книги.	
П3. Чтение	Пользователь открывает книгу в браузерном интерфейсе чтения; система фиксирует и синхронизирует позицию чтения.	Прогресс чтения сохранён; пользователь может вернуться к месту прерывания.
П4. Социальное взаимодействие	Пользователь выставляет оценку, оставляет комментарий, добавляет книгу в избранное или закладки.	Зафиксирована пользовательская активность; влияет на рекомендации.
П5. Интеллектуальный поиск	Пользователь обращается к диалоговому ассистенту или запрашивает персональные рекомендации.	Получен ответ по содержанию книги с источниками или подборка рекомендованных произведений.
П6. Администрирование	Администратор загружает книгу и обложку, создаёт карточку произведения, инициирует подготовку контента для AI.	Новое произведение становится доступно в каталоге и может использоваться в интеллектуальных сценариях.

Диаграмма прецедентов, отражающая указанные пользовательские сценарии, приведена ниже.

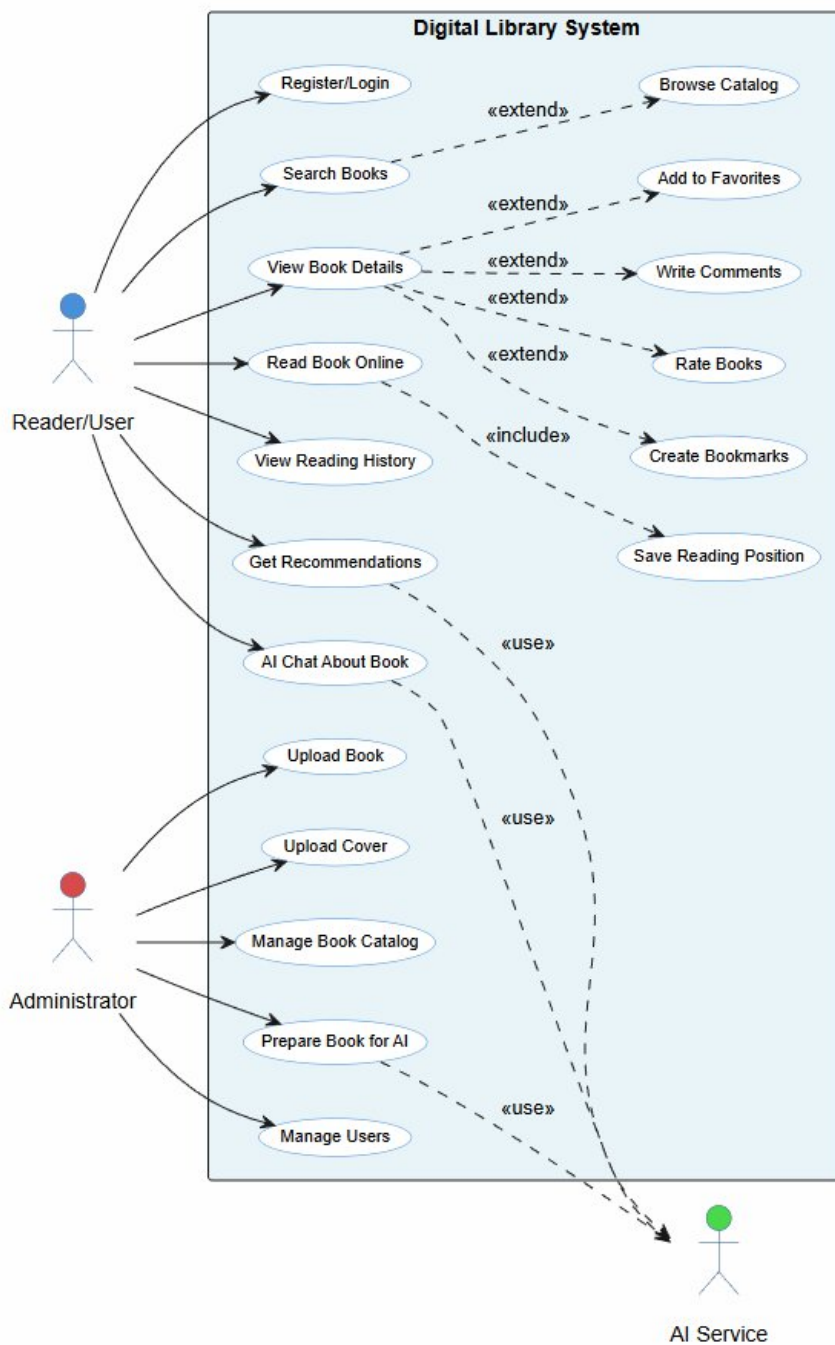


Рисунок 1.1 — Диаграмма прецедентов (Use Case) системы Digital Library

1.4.2 Архитектура микросервисной системы

Система реализована на основе микросервисной архитектуры с разделением функциональности по доменам [1,5]. Компонентная модель включает шесть основных микросервисов, инфраструктурные зависимости (PostgreSQL, Redis [11], Qdrant [16]) и веб-клиент.

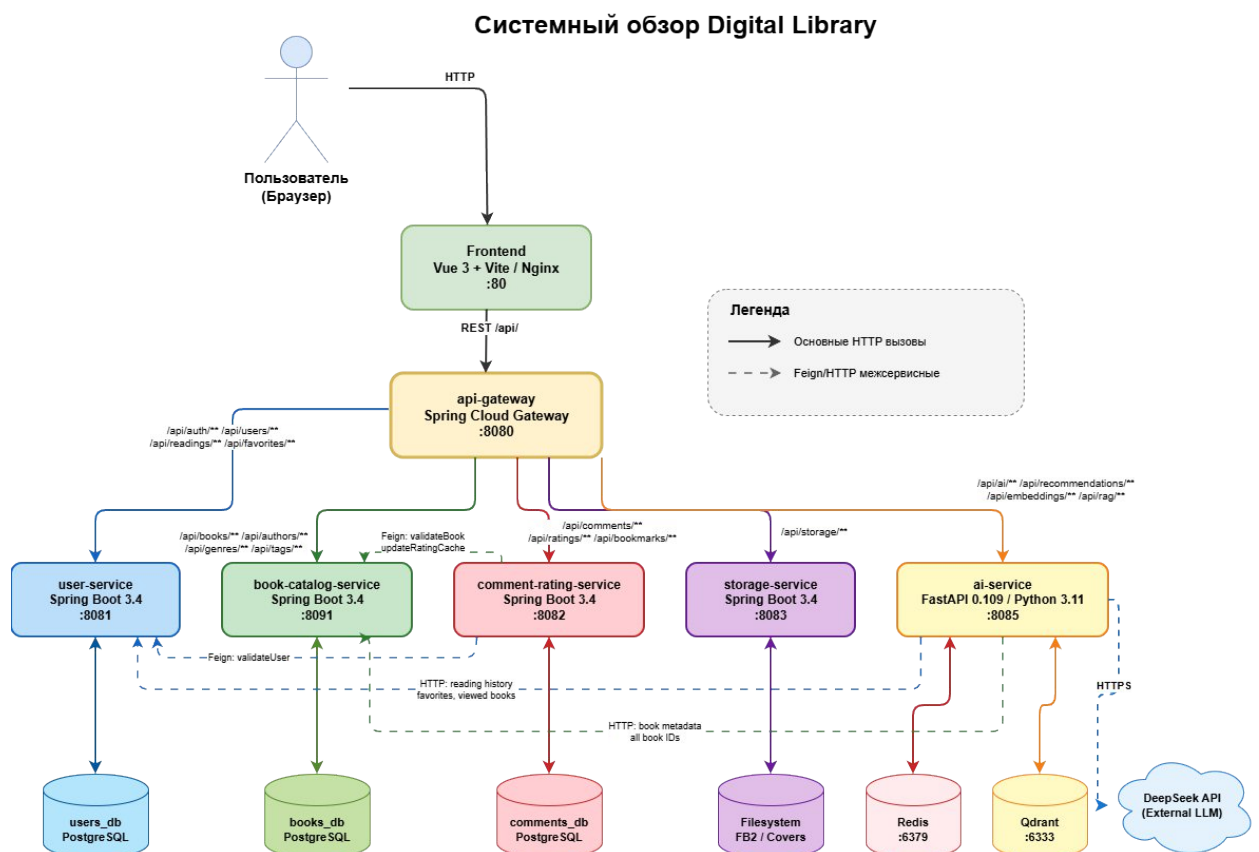


Рисунок 1.2 — Архитектурная схема взаимодействия компонентов системы

Функциональное разделение выполнено следующим образом: user-service — управление учётными записями, аутентификация, авторизация, история чтения, избранное; book-catalog-service — хранение и выдача метаданных книг, авторов, жанров, тегов, поиск и фильтрация; comment-rating-service — обработка комментариев, оценок и закладок произведений; storage-service — хранилище и потоковая передача файлов книг и обложек; ai-service — интеллектуальная обработка запросов на основе RAG, рекомендационный engine; api-gateway — единая точка входа для клиентских запросов, маршрутизация. Распределение функциональности по микросервисам и архитектурные уровни представлены в таблице 1.6.

Таблица 1.6 — Архитектурные уровни системы

Архитектурный уровень	Компоненты	Назначение
Клиентский	frontend (Vue 3 + Vite + Nginx)	Веб-интерфейс, маршрутизация, управление

Архитектурный уровень	Компоненты	Назначение
		состоянием клиента.
Входной	api-gateway (Spring Cloud Gateway)	Единая точка входа, маршрутизация по путям, CORS-контроль, ограничение по частоте запросов.
Доменный	user-service, book-catalog-service, comment-rating-service, storage-service, ai-service	Реализация бизнес-логики по функциональным зонам ответственности.
Данных	PostgreSQL (3 экз.), Redis, Qdrant	Транзакционные БД, кэш для сессий и рекомендаций, векторный поиск.

1.4.3 Проектирование API Gateway и межсервисного взаимодействия

На уровне проектирования API Gateway определён как единственная точка входа для всех клиентских запросов [6]. Маршрутизация строится по префиксу /api с направлением трафика к соответствующим upstream-сервисам. Такое решение обеспечивает единый контракт API для клиента, централизованное управление безопасностью (CORS, Rate Limiting) и независимое масштабирование компонентов. В качестве реализации выбран Spring Cloud Gateway.

Межсервисное взаимодействие проектируется на основе синхронных HTTP/REST-вызовов с декларативными клиентами (OpenFeign). Для устойчивости к частичным отказам предусмотрен паттерн Circuit Breaker (Resilience4j): при превышении порога ошибок обращения к недоступному сервису временно блокируются, предотвращая каскадную деградацию.

1.4.4 Проектирование базы данных

Проектирование данных выполнено по принципу «одна база на сервис». Это решение уменьшает связанность между доменами и позволяет развивать каждую часть системы независимо. В результате сформированы три транзакционные базы PostgreSQL, векторное хранилище Qdrant для семантического поиска и кэш-хранилище Redis для сессий, рекомендаций и истории диалогов.

В `users_db` хранятся данные пользователей, ролей, сессий чтения, избранного и истории просмотров. В `books_db` хранится каталог книг и связанная классификация (авторы, жанры, теги), а также денормализованные поля рейтинга книги для ускорения выдачи. В `comments_ratings_db` хранится пользовательская обратная связь: комментарии, оценки, закладки.

ER-диаграммы баз данных приведены ниже.

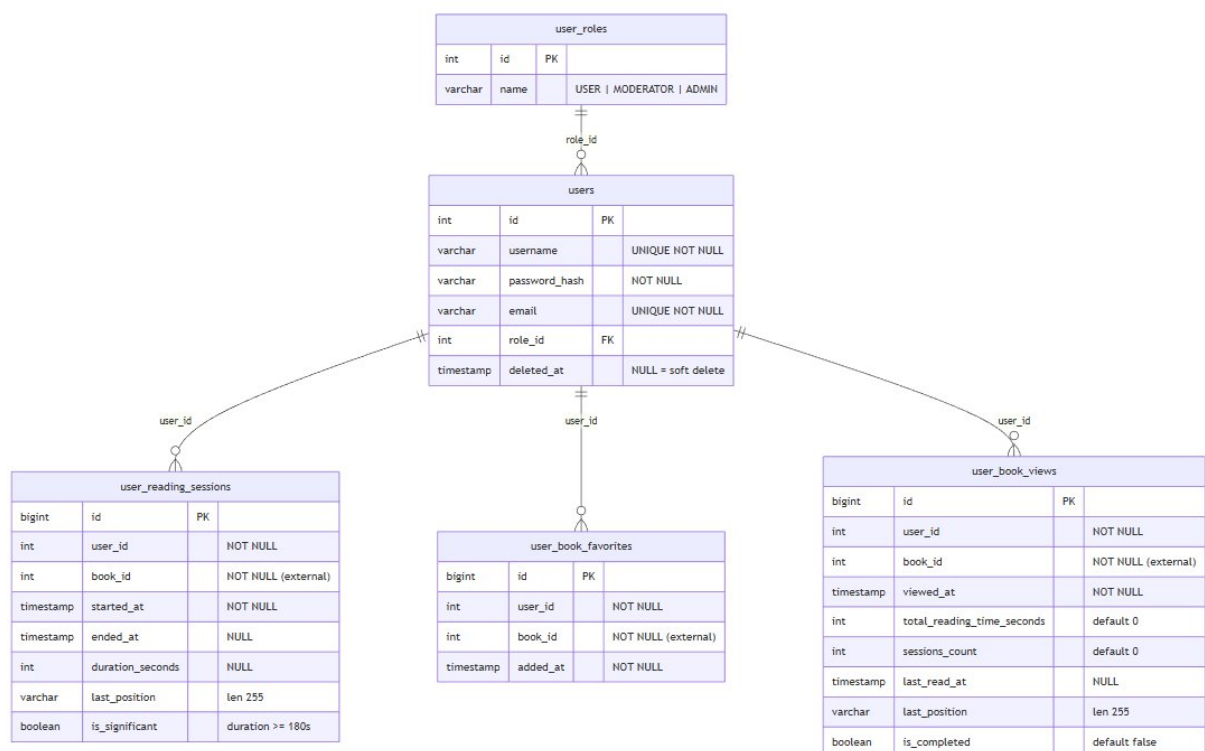


Рисунок 1.3 — ER-диаграмма базы данных `users_db`

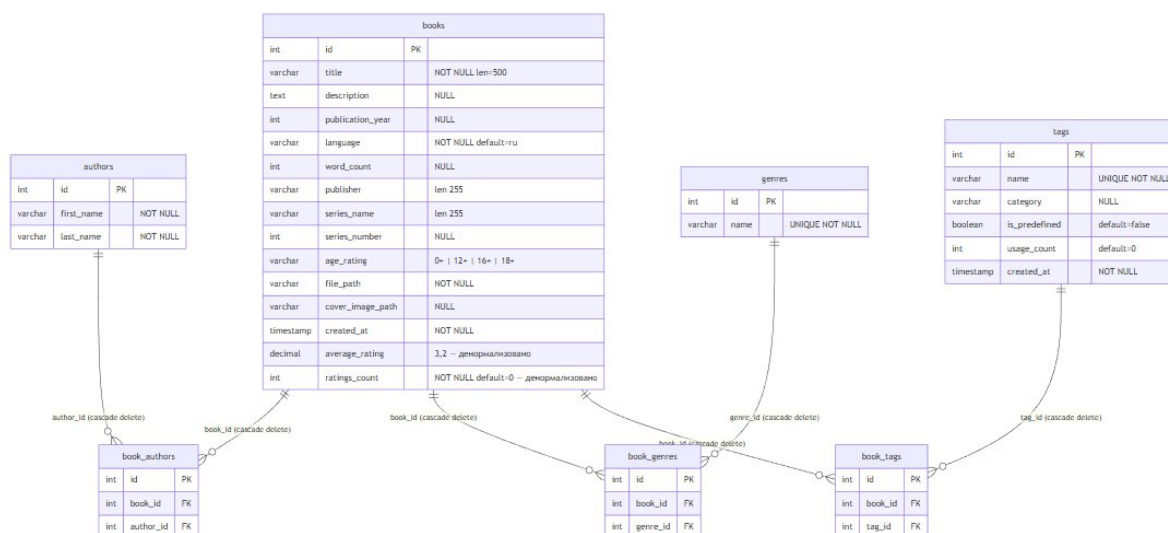


Рисунок 1.4 — ER-диаграмма базы данных books_db

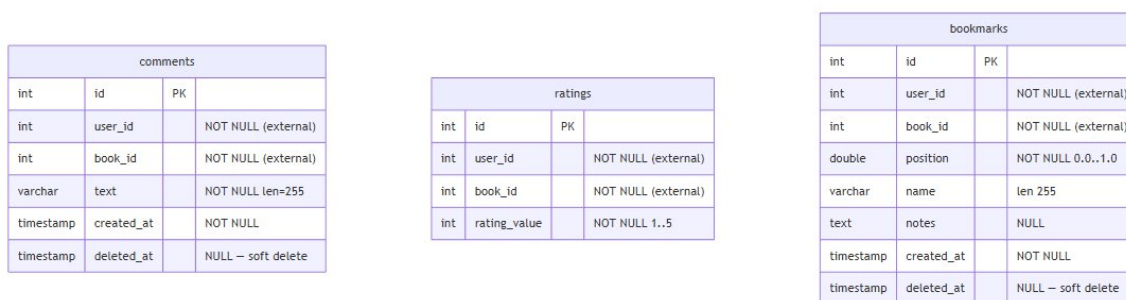


Рисунок 1.5 — ER-диаграмма базы данных comments_ratings_db

Таким образом, выбранная стратегия разделения данных по доменам обеспечивает слабую связанность микросервисов и позволяет масштабировать хранилища независимо.

1.4.5 Проектирование AI-контура

Для интеллектуальных сценариев архитектурно выделены два специализированных хранилища: Qdrant — для векторного поиска и рекомендаций, Redis — для кэша и сессионного контекста.

AI-сервис проектируется как самостоятельный микросервис с двумя контурами: RAG-контур (классификация запроса → поиск фрагментов →

генерация ответа LLM) и контур рекомендаций (сигналы пользователя → поиск похожих книг → персональная подборка).

Таблица 1.8 — Проектируемые компоненты AI-контура и их роль

Компонент AI-контура	Назначение в системе
Qdrant	Векторный поиск по смыслу для RAG-ответов и рекомендаций.
Redis	История диалога, кэш рекомендаций и оперативные данные AI-сценариев.
Классификатор намерений	Классификация пользовательского запроса и извлечение структуры запроса.
Модуль поиска и генерации (RAG)	Гибридный поиск релевантных фрагментов и подготовка контекста ответа.
Рекомендательный модуль	Персональные, похожие и популярные рекомендации на основе пользовательских сигналов и эмбедингов.
Менеджер контекста диалога	Управление контекстом диалога в рамках сессии пользователя.

1.4.6 Выбор технологического стека

Стек выбран по слоям: Java 21 + Spring Boot 3.4 (транзакционные сервисы) [7], Vue 3 [9] + Vite (фронтенд), Python 3.11 + FastAPI [8] (AI/RAG — богатая NLP-экосистема).

Таблица 1.9 — Обоснование выбора технологического стека

Технологический блок	Выбранные технологии	Причина выбора
Backend (транзакционный контур)	Java 21, Spring Boot 3.4, Spring Data JPA, Spring Security	Устойчивый стек для доменной логики, безопасности и интеграции с PostgreSQL.
API-вход и маршрутизация	Spring Cloud Gateway, Resilience4j	Единая точка входа, удобная маршрутизация и защитные механизмы на границе системы.
Frontend	Vue 3, Vite, Vue Router	Быстрая разработка SPA и удобная организация

Технологический блок	Выбранные технологии	Причина выбора
		пользовательских потоков.
AI-сервис	Python 3.11, FastAPI, sentence-transformers	Практичная реализация NLP/RAG и рекомендаций в экосистеме Python.
Хранилища	PostgreSQL, Qdrant, Redis [11]	Разделение транзакционных, векторных и кэшируемых данных по их природе.
Развёртывание	Docker Compose [12], Nginx (frontend)	Воспроизводимый запуск полного контура в единой среде.

2 Реализация системы

Реализация основана на микросервисной архитектуре с API Gateway. Стек: Spring Boot (Java-сервисы), FastAPI (AI), PostgreSQL (транзакционные данные), Redis (кэш/сессии), Qdrant (векторный поиск/RAG).

2.1 Организация репозитория и развёртывания

Проект разделён на директорию services (Maven-модули микросервисов с Dockerfile и ИИ модуль), модуль shared/common-models (общие JWT-утилиты, классы исключений) и клиентскую SPA на Vue 3.

Развёртывание выполняется через Docker Compose, что обеспечивает воспроизводимость конфигурации в различных окружениях. Файл docker-compose.yml поднимает все компоненты системы: PostgreSQL, Redis, Qdrant, пять Java-микросервисов, Python AI-сервис и фронтенд-приложение с Nginx.

Автоматизация сборки реализована через два CI/CD-конвейера в GitHub Actions: проверка при Push в ветки разработки и production-развёртывание при обновлении ветки main.

2.2 Реализация аутентификации и контроля доступа

Контур аутентификации реализован в user-service на базе Spring Security и JWT. При регистрации выполняется проверка уникальности учётных данных, хеширование пароля через BCrypt и запись учётной записи; при успешном входе формируется JWT-токен.

Верификация токена централизована в shared/common-models. Payload содержит идентификатор и роль пользователя (ROLE_USER/MODERATOR/ADMIN); подпись — HMAC с ограниченным сроком действия.

JwtAuthenticationFilter на уровне Gateway извлекает токен из заголовка Authorization или cookie и устанавливает контекст безопасности, исключая дублирование проверок в сервисах.

Схема процесса аутентификации приведена на рисунке 2.1.

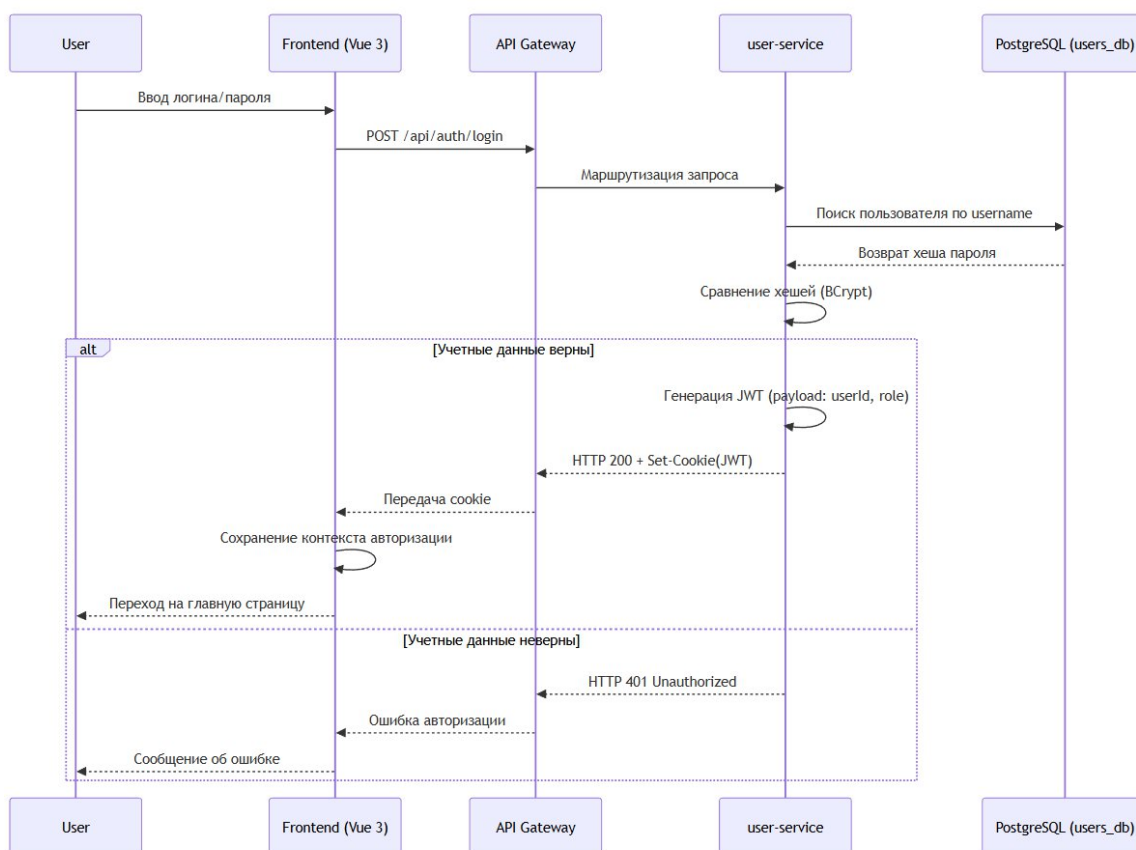


Рисунок 2.1 — Схема аутентификации пользователя в системе

2.3 Реализация каталога книг и классификаторов

Каталог реализован в book-catalog-service. REST API предоставляет операции для получения списков книг с пагинацией, загрузки информации о конкретном произведении, поиска по названию, автору или жанру с расширенными фильтрами.

На уровне данных реализована модель, включающая основные сущности: книги, авторы, жанры и теги. Связи между сущностями выражены через таблицы-связи многие-ко-многим, что позволяет одному произведению иметь несколько авторов, жанров и пользовательских меток.

2.4 Реализация социального контура: комментарии, оценки и закладки

Социальный контур реализован в comment-rating-service с контроллерами CommentController, RatingController и BookmarkController, каждый реализует полный CRUD.

Для комментариев реализована поддержка программного удаления (soft-delete), что сохраняет историю без физического удаления записей. Модерационные операции выделены в отдельные endpoint'ы и ограничены ролевыми проверками.

Перед записью система проверяет существование пользователя и книги через HTTP. Чтение комментариев и оценок открыто, создание записей требует JWT.

Таблица 2.1 — Межсервисные интеграционные сценарии

Интеграционный сценарий	Участвующие сервисы	Назначение
Проверка пользовательских операций	comment-rating-service → user-service	Подтверждение существования пользователя перед записью комментария/оценки.
Проверка связи с книгой	comment-rating-service → book-catalog-service	Подтверждение существования книги перед записью пользовательской активности.
Синхронизация рейтингов	comment-rating-service → book-catalog-service	Обновление average_rating и ratings_count в каталоге.
Подготовка AI-данных	ai-service → book-catalog-service / storage-service / user-service	Получение метаданных книг, файлов и пользовательских сигналов для рекомендаций.

2.5 Реализация хранения и потоковой передачи файлов

Контур управления файлами выделен в storage-service. API предоставляет загрузку и получение файлов; при загрузке валидируется расширение и проверяются права доступа (только администраторы).

Для электронных книг поддерживается формат FB2 с соответствующим типом MIME (application/x-fictionbook+xml). При выдаче файла система может использовать как встроенное отображение в браузере, так и принудительное

скачивание. Загрузка и удаление файлов ограничены ролевыми проверками (только модераторы и администраторы).

2.6 Реализация интеллектуального ассистента и RAG-контур

Интеллектуальный контур реализован в виде самостоятельного микросервиса. При запуске инициализируются подключения к Qdrant, Redis и моделям машинного обучения: рекомендательная модель (ru-en-RoSBERTa) загружается при старте, RAG-модель (USER bge-m3) — отложено при первом обращении, что снижает начальное потребление памяти.

Точка входа чат-API принимает запрос с идентификатором сессии, проверяет лимит обращений и возвращает структурированный ответ: тип намерения, текст, источники, рекомендации и `session_id`.

2.6.1 Процесс индексирования и поиска в RAG-пайплайне

Индексирование включает: разбиение текста на фрагменты с сохранением метаданных (книга, глава, позиция), нормализацию текста и расчёт длины в токенах.

Для каждого фрагмента вычисляются плотные векторные эмбединги и формируется разреженный BM25-индекс [14]. Полученные представления с метаданными (идентификаторы, позиция, превью, авторы, жанры, рейтинг) размещаются в векторной базе данных.

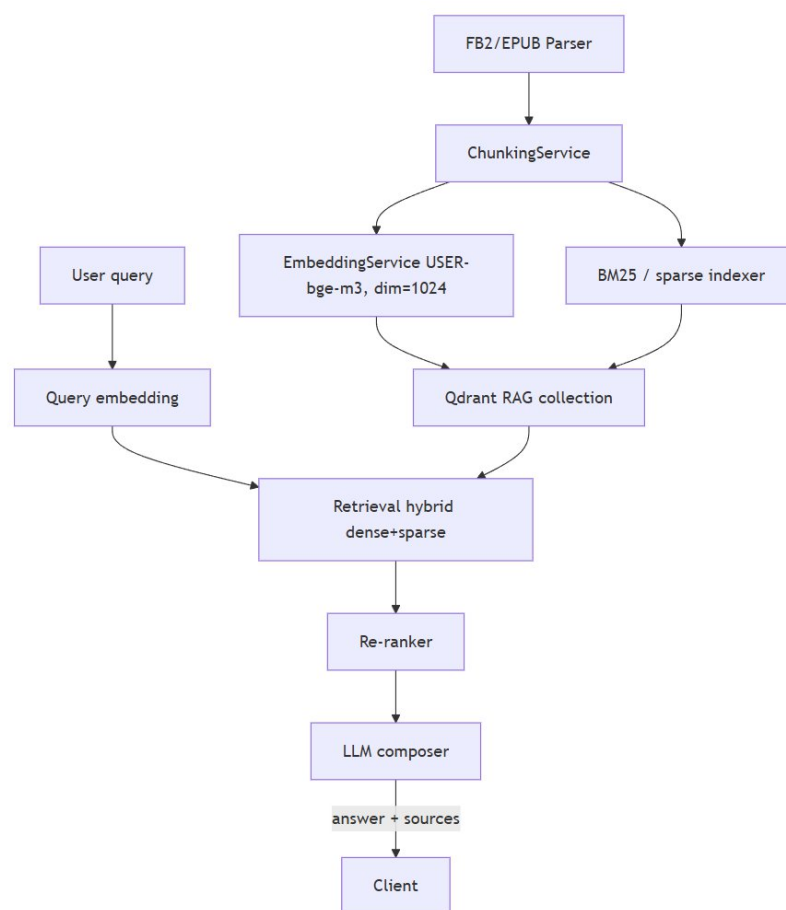


Рисунок 2.2 — Схема процесса индексирования и поиска в RAG-пайплайне

При поступлении запроса выполняется гибридный поиск: семантический по плотному вектору и лексический по разреженному индексу. Результаты объединяются и переранжируются с учётом скорингового балла и качественных характеристик источника.

Подготовленный контекст и системные инструкции передаются в языковую модель, которая генерирует текстовый ответ. Итоговый результат возвращается клиенту вместе со списком источников (provenance), подтверждающих достоверность сформированного ответа.

2.6.2 Алгоритм работы чат-ассистента

Чат-ассистент реализован в виде оркестратора, координирующего взаимодействие между подсистемами классификации намерений, разрешения контекста, поиска и генерации ответов. Ниже представлено поэтапное описание алгоритма обработки пользовательского сообщения.

Шаг 1. Восстановление состояния. Из кэша извлекается история диалога и закреплённый книжный контекст сессии, позволяющий обрабатывать уточняющие вопросы без повторного указания книги.

Шаг 2. Классификация намерения. Подсистема классификации определяет тип запроса (вопрос по содержанию, поиск цитаты, рекомендации или общий вопрос), извлекает нормализованную форму запроса и упомянутые сущности (названия, авторы, жанры).

Шаг 3. Разрешение книжного контекста. Упомянутые названия и авторы преобразуются во внутренние идентификаторы произведений; при отсутствии явных ссылок используется закреплённый контекст сессии.

Шаг 4. Диспетчеризация. Для вопросов по содержанию — RAG-пайплайн (гибридный поиск, расширение контекста, генерация ответа); для цитат — поиск с увеличенным числом результатов и специальной инструкцией; для рекомендаций — рекомендательный модуль (раздел 2.7).

Шаг 5. Формирование контекста и управление бюджетом токенов. Фрагменты упорядочиваются по релевантности, к значимым добавляются соседние для смысловой связности. При превышении бюджета токенов наименее релевантные фрагменты исключаются.

Шаг 6. Генерация ответа. Контекст, системная инструкция и запрос передаются в языковую модель; ответ возвращается с перечнем источников для верификации.

Шаг 7. Сохранение истории диалога. По завершении обработки запроса текущий ход диалога (сообщение пользователя и ответ ассистента) сохраняется в кэше под идентификатором сессии. Актуальный идентификатор сессии включается в состав ответного сообщения, что позволяет клиенту поддерживать непрерывность диалога при последующих обращениях.

Схема взаимодействия компонентов чат-ассистента в процессе обработки одного запроса представлена на рисунке ниже.

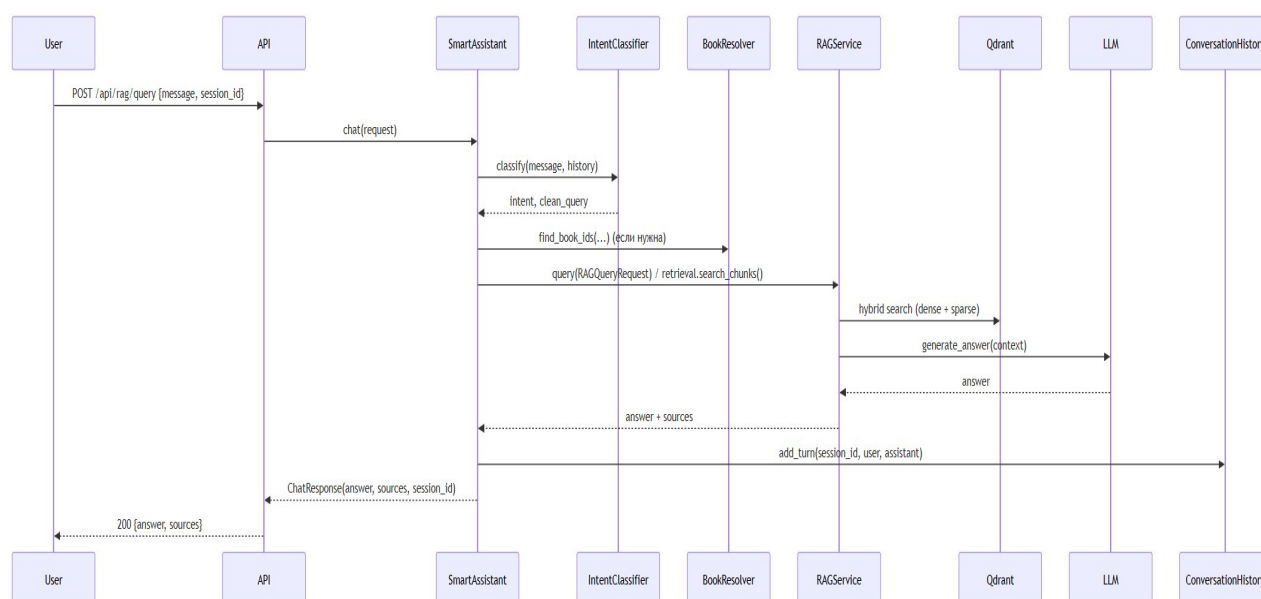


Рисунок 2.3 — Диаграмма последовательности обработки запроса чат-ассистентом

Параметры алгоритма (число фрагментов, глубина контекстного окна, веса поиска, бюджет токенов) вынесены в конфигурацию и не требуют изменения кода.

2.7 Реализация системы персональных рекомендаций

Рекомендательный контур реализован как отдельный компонент AI-сервиса. При наличии результата в кэше Redis используется кэшированная выдача, иначе запускается полный цикл обработки.

Пользовательский профиль формируется как взвешенное среднее эмбеддингов прочитанных/понравившихся книг. При отсутствии истории взаимодействий (холодный старт) используется стратегия популярных и новых книг без персонализации.

Гибридный поиск в Qdrant совмещает векторную близость с качественными сигналами (рейтинг, количество оценок); результаты переранжируются по MMR для разнообразия и кэшируются в Redis.

2.8 Реализация клиентской части

Клиентская часть — SPA на Vue 3 с Vite. Guard-функции маршрутизатора делят страницы на открытые и защищённые (JWT-токен).

Основные экраны: каталог с фильтрацией, карточка книги (метаданные, похожие, комментарии, чтение, оценка), профиль и панель администратора.

Скриншоты основных экранов клиентского интерфейса приведены в приложении А.

2.9 Интеграция клиента с серверной частью

Интеграция централизована в базовом HTTP-клиенте, который автоматически добавляет JWT [13] в заголовок Authorization. Все запросы направляются через /api на API Gateway.

Для сценариев, требующих сохранения контекста (например, длительный диалог с AI-ассистентом), фронтенд передаёт идентификатор сессии в параметрах запроса, что позволяет серверной части восстановить историю и контекст разговора.

2.10 Реализованные паттерны пользовательского опыта

Разработанная система реализует несколько практических решений для повышения удобства пользовательского опыта. Во-первых, для критичных экранов реализованы явные состояния загрузки и обработки ошибок. Во-вторых, компонент чата с AI-ассистентом имеет плавающую кнопку для открытия/закрытия, автоматическое прокручивание к последнему сообщению и индикатор обработки ответа. В-третьих, интерфейс чтения реализован как отдельное окно с визуальным контуром, ориентированным на длительное чтение текста.

Карточка книги в каталоге объединяет все связанные операции в одном месте: отображение информации о произведении, кнопка для запуска чтения, кнопка добавления в избранное, возможность выставить оценку, просмотр комментариев и список похожих книг.

2.11 Детали реализации схем данных

Схемы данных спроектированы согласно принципу «одна база на сервис» (раздел 1.3.4). Ниже описаны ключевые реализационные решения, не отражённые на ER-диаграммах.

Ключевое решение — денормализация рейтингов: поля `average_rating` и `ratings_count` хранятся в таблице книг и обновляются посредством синхронного HTTP-вызова из `comment-rating-service` при изменении оценок, что исключает ресурсоёмкие агрегирующие запросы при открытии каталога.

3 Тестирование и оценка результатов

Тестирование охватывало функциональную проверку, нагрузочное тестирование и анализ эксплуатационных метрик, что позволило оценить корректность бизнес-логики и устойчивость системы при росте нагрузки.

3.1 Цели и методология тестирования

Применён комплексный подход: функциональное тестирование ключевых сценариев, нагрузочное тестирование с k6 [17] и мониторинг метрик через Prometheus [18] и Grafana [19].

Тестирование проводилось на учебном стенде со следующей конфигурацией: 4 vCPU, 8 GB RAM, SSD и контейнеризированное развёртывание всех сервисов через Docker Compose.

3.2 Сценарии нагрузочного тестирования

Были определены два независимых сценария нагрузки, описанных в таблице 3.1.

Таблица 3.1 — Сценарии нагрузочного тестирования

Сценарий	Executor	RPS	Duration	VUs	Цель
catalog	constant-arrival-rate	50	5 мин	100	Оценка производительности транзакционного контура
ai	constant-arrival-rate	5	5 мин	20	Оценка латентности AI-пайплайна (RAG + LLM)

Сценарий catalog: листинг книг с фильтрацией, детальная карточка, загрузка файлов, поиск. Сценарий ai: рекомендации, проверка доступности и диалоговый RAG-запрос.

Пороговые значения прохождения теста: `http_req_failed` < 2% (доля ошибок 5xx); `checks` > 95% (доля успешных валидаций ответов).

3.3 Результаты нагрузочного тестирования

3.3.1 Производительность транзакционного контура

Таблица 3.2 — Метрики транзакционного контура

Метрика	Значение	Условие	Комментарий
p95 latency	180–240 мс	50 RPS, 1000 книг	Стабильная работа без деградации
p99 latency	420 мс	Пиковые запросы	Влияние сборки сложных агрегатов
Error rate (5xx)	0,3%	Общая нагрузка	Единичные таймауты при старте контейнеров
Throughput	48–52 RPS	Целевая нагрузка	Система удерживает заданную интенсивность
Redis cache hit rate	72%	Рекомендации	Эффективное кэширование пользовательских сигналов

3.3.2 Производительность AI-контура

Таблица 3.3 — Метрики AI-контура

Метрика	Значение	Условие	Комментарий
p95 latency (RAG)	1,8–8,4 с	5 RPS, контекст ~5 фрагментов	Основное время расходуется на генерацию LLM
p95 latency (рекомендации)	320 мс	При cache miss	Векторный поиск в Qdrant и скоринг кандидатов
Cache hit rate (рекомендации)	68%	Повторные запросы	Снижение нагрузки на Qdrant и LLM
Error rate	1,1%	Общая нагрузка	Таймауты при перегрузке LLM-эндпоинта

3.3.3 Ресурсная утилизация

Таблица 3.4 — Ресурсная утилизация сервисов

Сервис	CPU (средн.)	Heap Memory	DB Connections
api-gateway	12%	280 МБ	—
book-catalog-service	18%	420 МБ	8/20
user-service	9%	310 МБ	5/20

Сервис	CPU (средн.)	Heap Memory	DB Connections
ai-service	45%	1,1 ГБ	—
PostgreSQL (books_db)	22%	1,8 ГБ	35/100
Redis	6%	420 МБ	—
Qdrant	38%	2,3 ГБ	—

Наиболее ресурсоёмкий компонент — AI-сервис, изолированный в отдельном контейнере с кэшированием в Redis. Дашборды Grafana приведены в приложении А.

3.4 Анализ узких мест и компромиссов

По результатам мониторинга выявлены следующие особенности поведения системы.

- 1) Латентность AI-запросов (1,8–8,4 с) обусловлена последовательным retrieval в Qdrant и генерацией LLM без потоковой выдачи.
- 2) CPU Qdrant растёт до 38% при параллельных запросах. Основная компенсация нагрузки — кэширование результатов поиска в Redis (cache hit rate 68%), что снижает фактическое число обращений к векторной БД. RRF [15] является алгоритмом слияния ранжирующих списков и не является средством снижения нагрузки на CPU.
- 3) Пул соединений с БД (HikariCP [10], maximum-pool-size=20) используется на 40% при пиковой нагрузке — достаточно для учебного стенда.
- 4) Отсутствует распределённая трассировка; рекомендуется внедрение OpenTelemetry.

3.5 Функциональное тестирование и валидация RAG

Проведена ручная валидация качества ответов AI-ассистента по критериям: релевантность фрагментов, корректность пересказа, отсутствие вымышленных сведений и наличие ссылок на источники.

Таблица 3.5 — Результаты валидации RAG-ответов (20 тестовых запросов)

Критерий	Доля успешных ответов
Релевантность контекста	85%
Точность пересказа	78%
Наличие источников	92%
Отсутствие выдумок	81%

На основе 20 тестовых запросов установлено, что качество ответов является удовлетворительным для прототипа. Основные ошибки связаны с недостаточной детализацией контекста при сложных многозначных запросах.

3.6 Интеграция мониторинга в процесс разработки

Мониторинг на Prometheus/Grafana интегрирован в CI: конфигурация и дашборды (JSON) хранятся в репозитории. Настроены дашборды по латентности, RPS, ошибкам и ресурсам, а также правила оповещения в Alertmanager.

3.7 Итоговые выводы по разделу

Система выдерживает 50 RPS (транзакционный контур) и 5 RPS (AI); разделение контуров предотвратило каскадную деградацию, а латентность AI соответствует ожидаемым значениям для прототипа.

Для промышленной эксплуатации необходимы масштабирование AI-сервиса, расширение покрытия функциональными тестами.

ЗАКЛЮЧЕНИЕ

Современные электронные библиотеки трансформируются из простых каталогов метаданных в интеллектуальные среды для чтения и глубокой работы с контентом. Пользователи сталкиваются с проблемой информационной перегрузки: им требуется не просто доступ к файлам, но и инструменты для семантического поиска, персонализации и интерактивного взаимодействия с текстом. Разрабатываемая система «Digital Library» решает данную проблему, объединяя классический библиотечный функционал, социальное взаимодействие и возможности искусственного интеллекта в едином бесшовном пользовательском сценарии.

В ходе выполнения работы был проведен анализ предметной области и изучены существующие платформенные решения. На основе выявленных функциональных разрывов были сформулированы требования к системе и обоснован выбор микросервисного стека технологий. В результате выполнения работы была спроектирована и реализована масштабируемая веб-система «Digital Library». Разработанное решение включает в себя контуры JWT-аутентификации, управления каталогом книг, социального взаимодействия (комментарии, оценки, закладки) и хранения файлов. Особое внимание было уделено созданию интеллектуального модуля, реализующего гибридный RAG-пайплайн для диалогового поиска по содержанию книг и алгоритм персональных рекомендаций на основе векторных эмбедингов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ньюман С. Создание микросервисов. — СПб.: Питер, 2016. — 304 с.
2. Ричардсон К. Микросервисы: паттерны разработки и рефакторинга. — СПб.: Питер, 2019. — 544 с.
3. Kucev. Retrieval-Augmented Generation (RAG): глубокий технический обзор [Электронный ресурс] // Хабр. — 2025. — URL: <https://habr.com/ru/articles/931396> (дата обращения: 15.04.2026).
4. Lewis P., Perez E., Piktus A. и др. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [Электронный ресурс]. — 2020. — URL: <https://arxiv.org/abs/2005.11401> (дата обращения: 15.04.2026).
5. Serverspace. Микросервисная архитектура, её паттерны проектирования и особенности [Электронный ресурс] // Хабр. — 2022. — URL: <https://habr.com/ru/companies/serverspace/articles/692916> (дата обращения: 08.04.2026).
6. Katherine_a. Что такое API Gateway: 10 главных функций и роль в архитектуре микросервисов [Электронный ресурс] // Хабр. — 2025. — URL: <https://habr.com/ru/articles/967830> (дата обращения: 11.04.2026).
7. Документация Spring Boot [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/current/reference/html> (дата обращения: 07.04.2026).
8. Документация FastAPI [Электронный ресурс]. — URL: <https://fastapi.tiangolo.com> (дата обращения: 09.04.2026).
9. Документация Vue.js 3 [Электронный ресурс]. — URL: <https://vuejs.org/guide/introduction> (дата обращения: 11.04.2026).
10. Документация HikariCP [Электронный ресурс]. — URL: <https://github.com/brettwooldridge/HikariCP> (дата обращения: 16.04.2026).
11. Документация Redis [Электронный ресурс]. — URL: <https://redis.io/docs> (дата обращения: 10.04.2026).

12. Документация Docker [Электронный ресурс]. — URL: <https://docs.docker.com> (дата обращения: 06.04.2026).
13. JSON Web Token Introduction [Электронный ресурс]. — URL: <https://jwt.io/introduction> (дата обращения: 13.04.2026).
14. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond // Foundations and Trends in Information Retrieval. — 2009. — Vol. 3, № 4. — P. 333–389.
15. Cormack G. V., Clarke C. L., Buettcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods // Proceedings of SIGIR 2009. — 2009. — P. 758–759.
16. Документация Qdrant [Электронный ресурс]. — URL: <https://qdrant.tech/documentation> (дата обращения: 08.04.2026).
17. Документация k6 [Электронный ресурс]. — URL: <https://grafana.com/docs/k6/latest> (дата обращения: 17.04.2026).
18. Документация Prometheus [Электронный ресурс]. — URL: <https://prometheus.io/docs/introduction/overview> (дата обращения: 12.04.2026).
19. Документация Grafana [Электронный ресурс]. — URL: <https://grafana.com/docs/grafana/latest> (дата обращения: 12.04.2026).

ПРИЛОЖЕНИЕ А

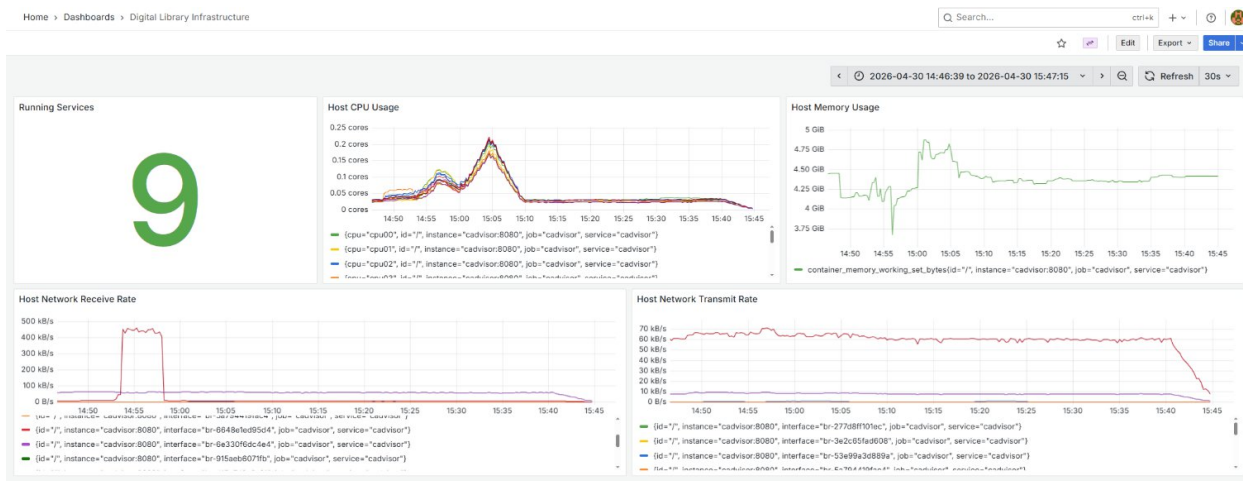


Рисунок А.1 — Дашборд Grafana: общая инфраструктура развёртывания

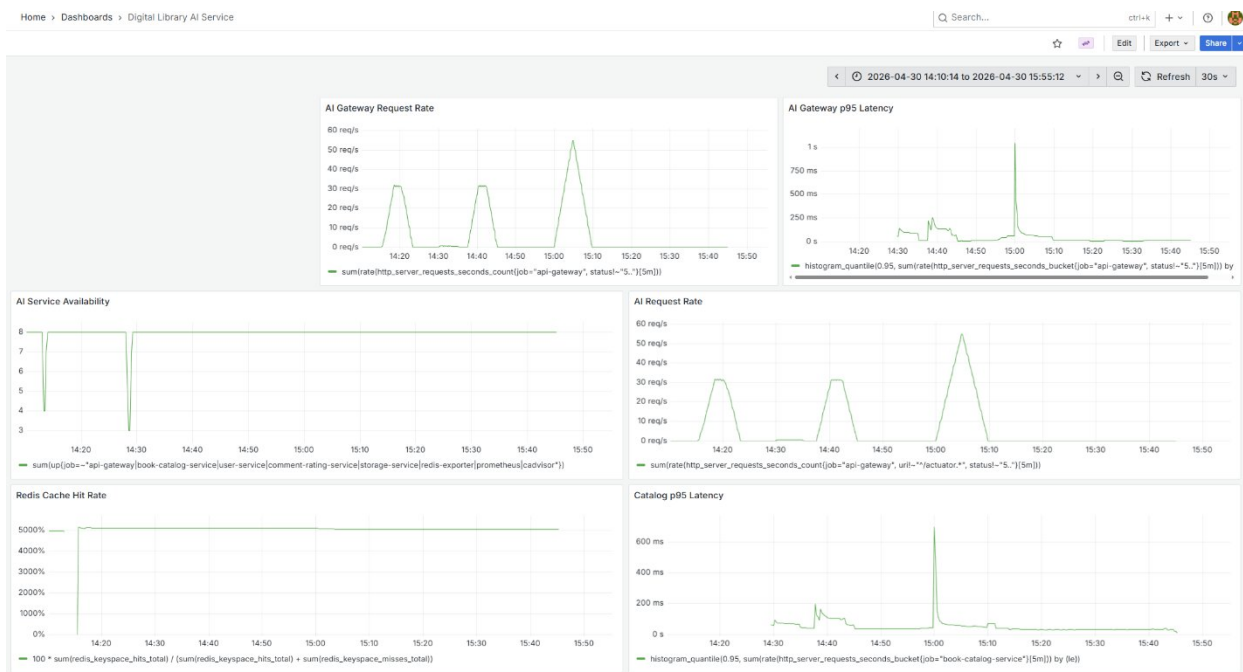


Рисунок А.2 — Дашборд Grafana: поведение AI-сервиса под нагрузкой

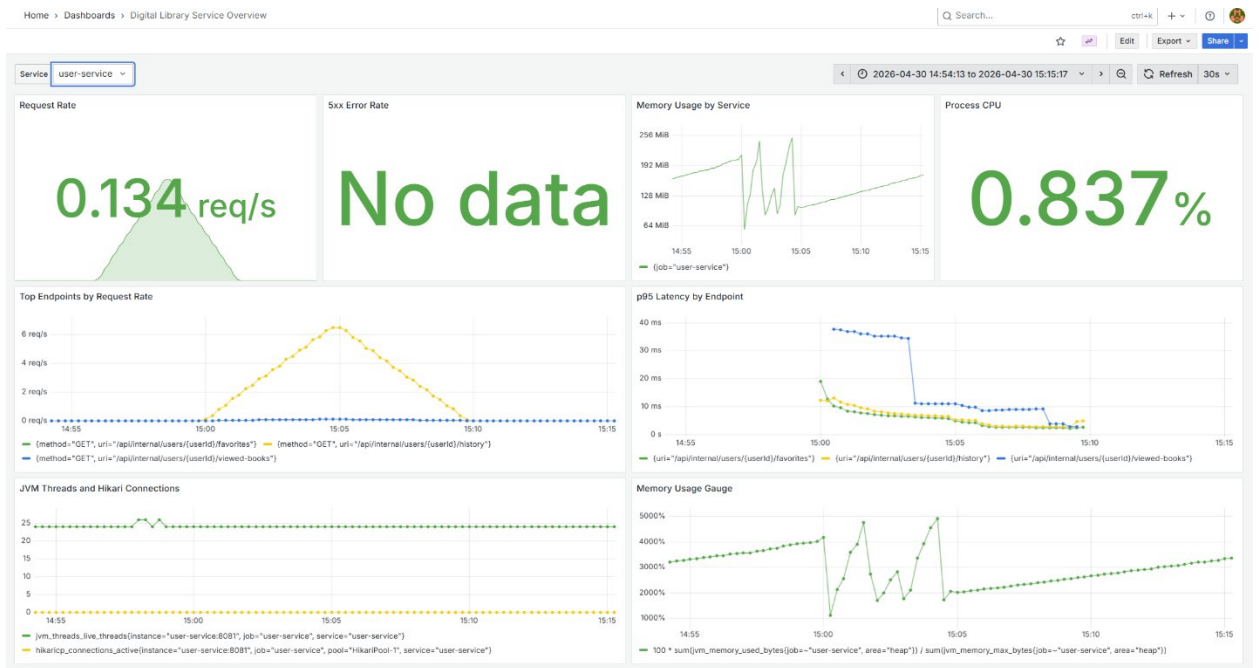


Рисунок А.3 — Дашборд Grafana: мониторинг user-service

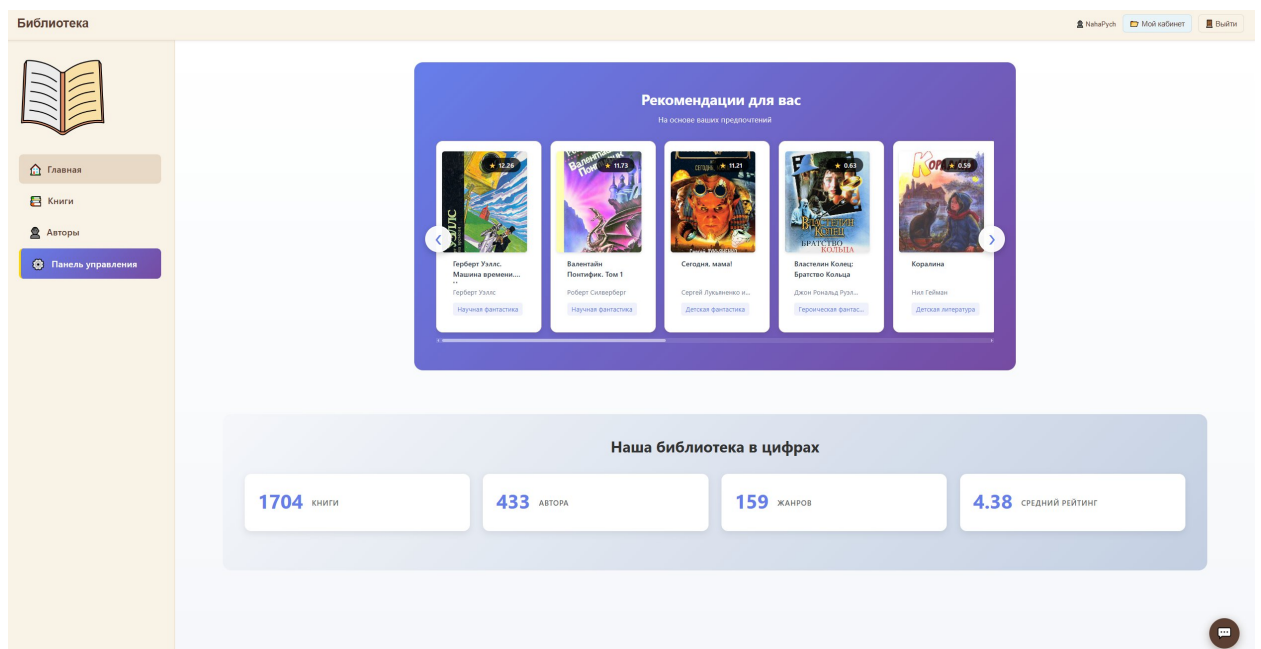


Рисунок А.4 — Главная страница с персональными рекомендациями

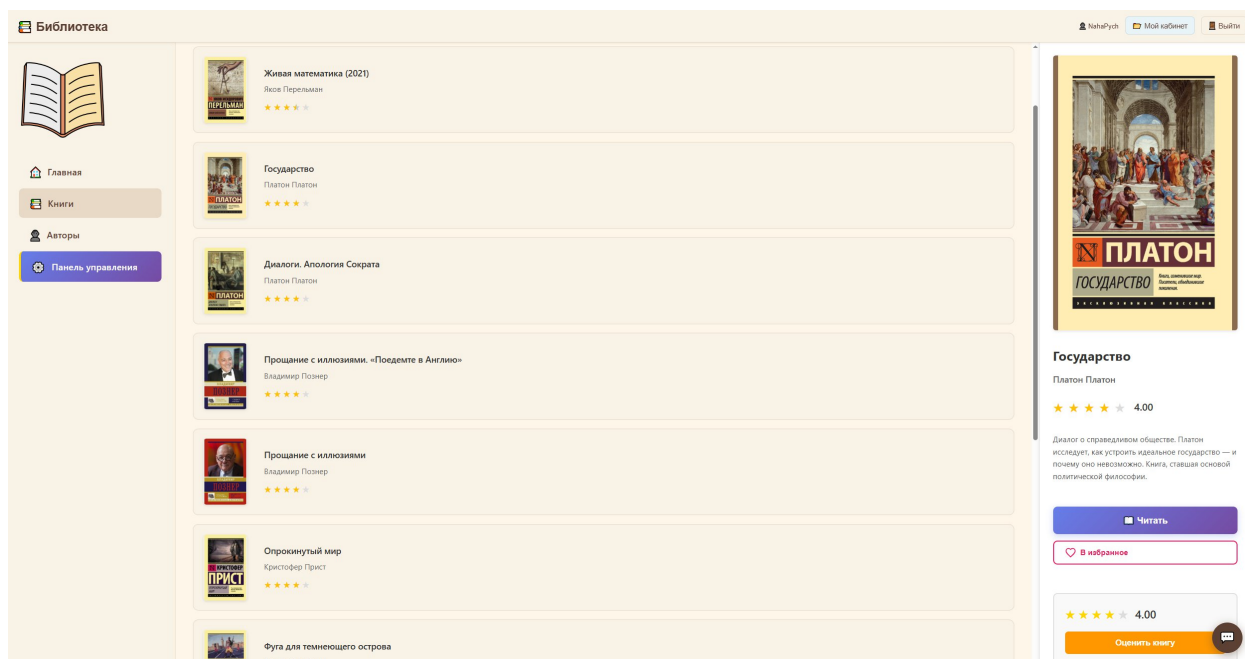


Рисунок А.5 — Экран каталога книг с панелью детальной информации



Рисунок А.6 — Интерфейс AI-ассистента в режиме рекомендаций